REPETITIONS (LOOPS)

CS1910

Computer Science I

CODE EXAMPLES

- https://colab.research.google.com/github/UPEI-CS/CS1910-Students/blob/main/lectures/07-Repetitions/CS1910_Repetitions_Workbook.ipynb

REPETITIONS

- We have seen and been using Repetitions since early in the course but not specifically in Python
- Repetitions (AKA) loops allow us to execute code multiple times
- When paired with concepts from computational thinking: *decomposition* and *patterns*
  - It allows us to write elegant efficient solutions that are easy to read and maintain

With a bit of experience

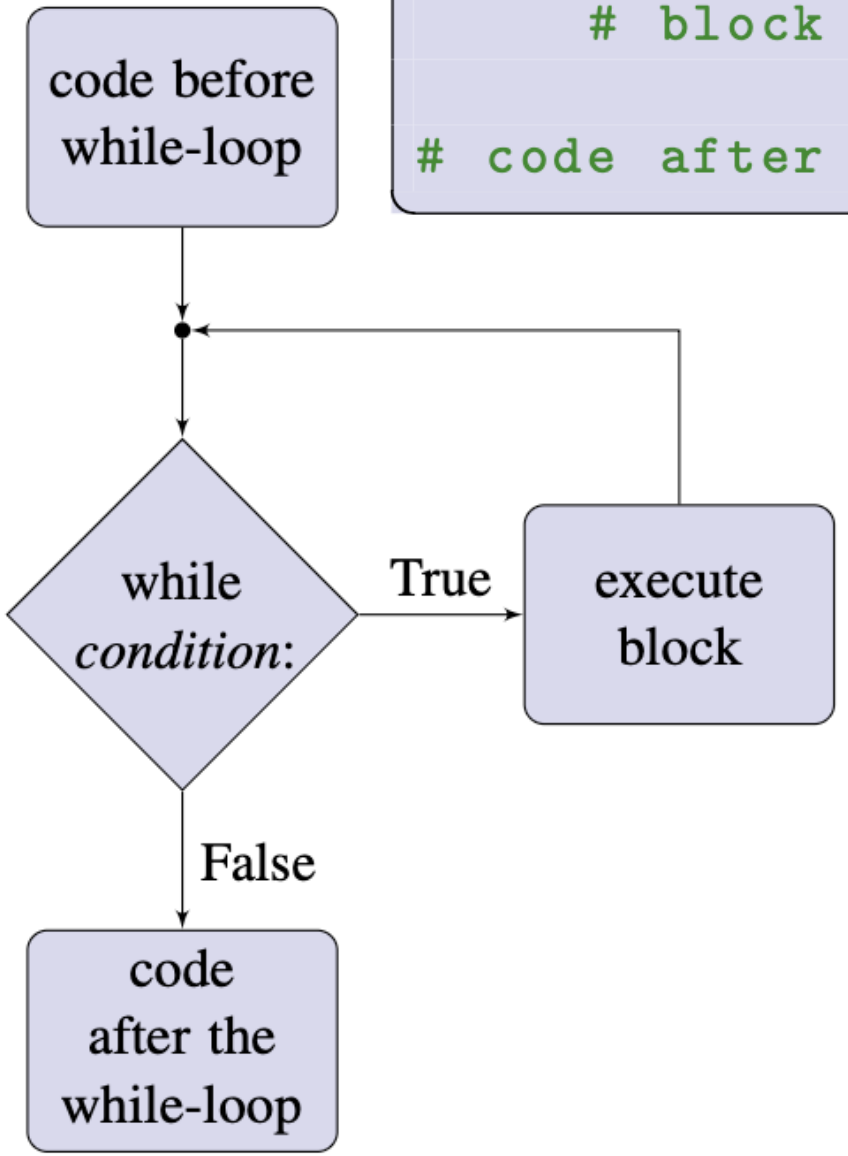
WHILE LOOPS

Controlled by a Boolean expression, execute specific code until the condition is False

```
# code before while-loop

while condition:
      # block (indented)

# code after the while-loop
```



QUICK HITTER

- What is the minimum number of times the code inside a while loop must execute?

```
# code before while-loop

while condition:
      # block (indented)

# code after the while-loop
```

QUICK HITTER

- What is the minimum number of times the code inside a while loop must execute?

```
# code before while-loop

while condition:
      # block (indented)

# code after the while-loop
```

No times! The condition might be initially False

WHILE LOOPS

Credit:Wikipedia



- Example: SENTINEL Loop

```
# Sentinel while loop example

#ensure the user has picked correctly
choice = int(input("Enter 1 for CS help or 2 for Math help"))
while choice != 1 and choice != 2:
    choice = int(input("Incorrect Input. Enter 1 for CS help or 2 for Math help"))

if choice == 1:
    print("For CS Help first turn off your computer and then turn it back on.")
else:
    print("For Math help first complete a skill testing question: Find x such that
```

A sentinel is something that watches for a specific occurrence

After the while loop we know that `choice` must be either 1 or 2

WHILE LOOP EXAMPLES

```python
# Sentinel While Loop Example

#calculate class average
value = input("Grade (DONE to Quit):")
sum_grades = 0
count = 0
while(value != "DONE"):
    sum_grades += int(value)
    count += 1
    value = input("Grade (DONE to Quit):")

if count > 0:
    print(f"Class average {(sum_grades / count):.1f}")
```

```python
# Sentinel While Loop Example
# Challenge modify this program to count how many guesses it takes

import random
secret = random.randint(1,10) #generates a random number between 1 and 10

guess = int(input("I'm thinking of a number between 1 and 10, guess it:"))

while(guess != secret):
    guess = int(input("Nope! Guess again: "))

print("Yay you guessed correctly!")

```

TRACING A WHILE LOOP

```
1 # while loop example:
2 # What does this code do
3
4 number = int(input("Enter a positive number:"))
5 reverse = 0
6
7 while number > 0:
8     last_digit = number % 10
9     reverse = reverse * 10 + last_digit
10    number = number // 10
11
12 print(reverse)
```

FOR LOOPS

```
for variable in sequence:
    # Block of code -- each time this block is repeated,
    # variable refers to the next item in the sequence.
    # Repetition stops after each item in the sequence has
    # been processed.
```

RANGE

- **Range** a sequence of integers starting from some integer **start**, continuing up to but not including some ending integer **stop,** advancing **step** integers at a time
- Built-in python function:

Start defaults to 0 and step to 1

```
range(stop)
range(start, stop[,step])
```

Step is optional, if omitted it defaults to 1

- The stop value is not included (exclusive of the stop)
- Used to with for loops to execute an exact number of times (counting loop)

RANGE EXAMPLES

```
range(0,5,1)      # the sequence 0, 1, 2, 3, 4
range(5)          # the sequence 0, 1, 2, 3, 4
range(-4, 4)      # the sequence -4, -3, -2, -1, 0, 1, 2, 3
range(0, 11, 2)   # the sequence 0, 2, 4, 6, 8, 10
range(2, -3, -1)  # the sequence 2, 1, 0, -1, -2
range(0, 5, 10)   # the sequence 0
```

FOR LOOPS WITH RANGE

FOR LOOP WITH RANGE

```
for i in range(N):
    # do something
```

```
for i in range(5):
    print(i)
```

EQUIVALENT WITHOUT LOOPS

```
i = 0
print(i)

i = 1
print(i)

i = 2
print(i)

i = 3
print(i)

i = 4
print(i)
```

OUTPUT

```
0
1
2
3
4
```

The loop executes 5 times

Remember: The end value passed to range is excluded

CONVENTIONS

- Typically, we avoid single letter variable names
  - However, it is quite common to use i, j, and k, … As the counting variable in a for loop
  - It is always a good idea to ask yourself if another name is more appropriate
- You may see the use of an _ as the variable name for use in a for loop
- This is a valid variable name but it indicates that the counting variable for the loop is not used in the loop.
  - i.e., you want to do something a specific amount of times but there is no regard for which iteration it is

SEQUENCES AND FOR LOOPS

- The only sequence so far (aside from the built-in range) we have seen is a string
- A *for* loop can process each of the letters in a string

```
name = "The Quick BroWn FoX jumpEd OveR tHe lazy DOG"
count = 0
for char in name:
    if char.isupper():
        count += 1
print(count)
```

NESTED LOOPS

- When loops are nested a few lines of code can create complex iterations

```
# nested loops to make a multiplication table
for i in range(1,11):
    for j in range(1,11):
        print(f"{i*j:4}", end="")
    print()
```

Belongs to outer loop

Pad with 4 spaces

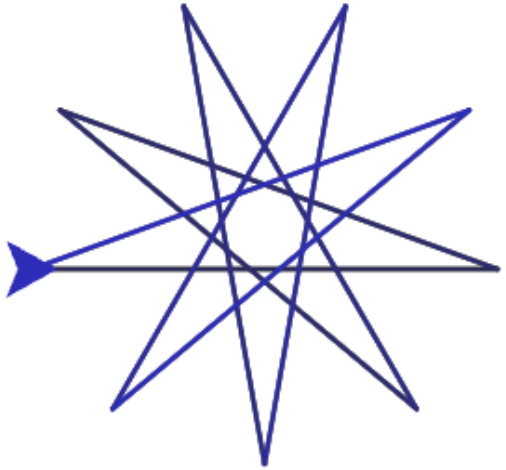
Instead of newline after the print use empty string

NESTED LOOPS WITH TURTLE DRAW

What shape does this create?

```
for j in range(9):
    for k in range(2):
        forward(100)
        right(200)
```

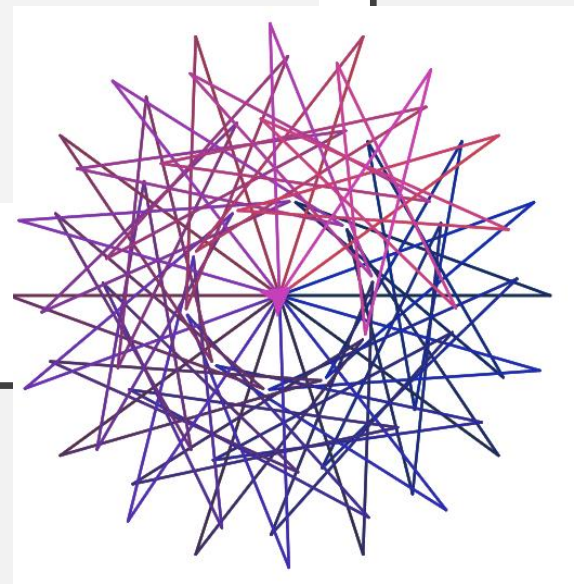
NESTED LOOPS WITH TURTLE DRAW



```
for j in range(9):
    for k in range(2):
        forward(100)
        right(200)
```

TRIPLE NESTED LOOP

```
for i in range(10): #spirals to draw
    color(255,0,0)
    for j in range(9): #each spiral has 9 arms
        color(i*25,50, j*25) #gets more red each spiral, each arm gets more blue
        for k in range(2): # each arm has 2 lines (out and back)
            forward(100)
            right(200)
    right(36) #360 / 10 is 36 degree offset for each spiral
```



BEWARE

- Infinite loops are loops that run forever
  - (sometimes by design … a web server listens for connections forever)
  - Often because we have a bug … forgotten to increment a counter, broken logic, etc
- Indentation is critical
  - Code blocks are denoted by indentation levels
- Variables can be declared and assigned inside code blocks (decision or repetition)
  - Is this efficient, is there a case where the variable is not declared because it was inside a loop and the loop didn't run

FIND A PARTNER

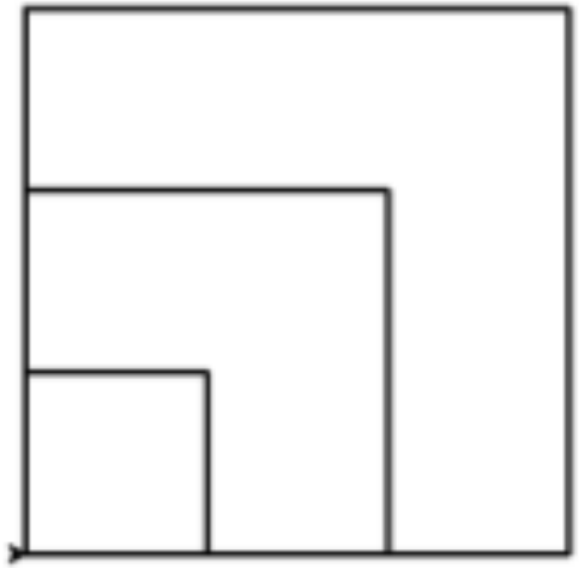
- Warm-up:
  - Modify the random guess While loop example so that it counts how many guesses the user took to get it right. Print out how many guesses
- Challenge:
  - Modify the random guess While loop example so that after they have guessed correctly it asks them if they want to play again. If they do not want to play again it outputs their average number of guesses to correctly complete the game.
- Think about:
  - If you played the game one thousand times what do you expect the average number of guesses to be?

FIND A PARTNER

Use colabturtleplus to draw the nested square shape

1. Decomposition
   - what are the basic operations we need to do?
   - don't forget our previous work drawing a single square
2. Patterns:
   - what are the patterns (or patterns within patterns)
3. Algorithm

   Outline in pseudo code your algorithm
4. Lastly – move onto the code



THROWING DARTS

Suppose you have a dart board with radius 1 and you place a square with side length 2 directly behind the dart board.

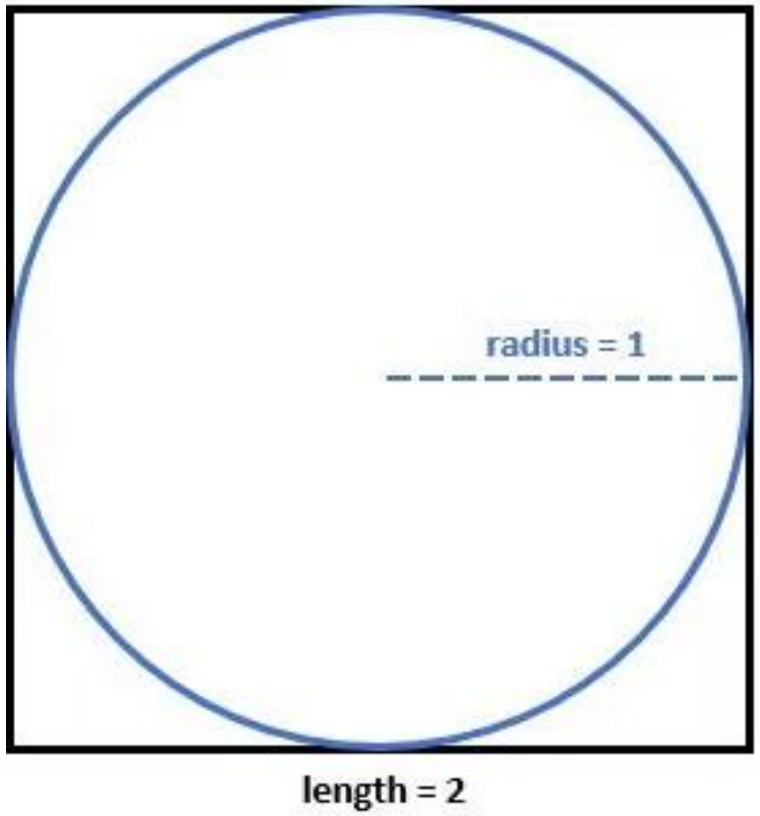
Consider that your friend throws darts at the board with equal chance it lands anywhere within the square

How many darts would land inside the circle?

What percentage of the area is covered by circle?

Thinking exercise

Cool simulation



SIMULATING PI

Remember your buddy throws darts with equal chance of landing anywhere in the square

percentage_circle = Area of circle / Area of square

$= \pi r^2 / (2r)^2$

$= \pi r^2 / 4r^2$

$= \pi / 4$

percentage hitting the circle = c / n ← Darts hitting the circle

so

$c / n \approx \pi / 4$ ← Total darts

Thus we can approximate π as 4c / n

ALGORITHM TO SIMULATE PI

Algorithm:

1. Set c = 0 (darts inside the circle) and n = 0 (total darts)
2. Repeat this many times:
    a. Generate a random x and a random y, each between 0 and 1
    b. Work out the distance from the origin: $\sqrt{(x^2 + y^2)}$
    c. If the distance is 1 or less, the dart is inside the circle: add 1 to c
    d. Add 1 to n
3. Print $4 \times c / n$ as the estimate for π

TRY AT HOME

1. Walk through the Sentinel Loops in the Try At Home Sentinels block.
2. Modify the guess game:
   - Make it so the guesses are random as well and are not user input.
   - Count how many guesses it takes to get it right.
   - Simulate 100 games.
   - Track and print the average number of guesses needed to
3. Complete the code to simulate pi.
4. Use loops to draw a nested circle.
5. Use loops to draw a triangle spiral

